

Введение

В современной индустрии видеоигр наблюдается значительный рост числа независимых разработчиков (indie-dev), которым требуется специализированная платформа для продвижения своих продуктов, коммуникации с аудиторией и организации технической поддержки. Существующие решения (Steam, Epic Games Store, itch.io) фокусируются преимущественно на дистрибуции, оставляя нишу комплексных инструментов поддержки и взаимодействия с пользователями практически незанятой.

Цель дипломного проекта заключается в разработке и реализации веб-платформы, объединяющей функционал каталога игр, системы подписок, технической поддержки через тикеты, публикации контента (новости, гайды) и социального взаимодействия (чат, комментарии, подписки на разработчиков).

На дипломный проект определены следующие задачи:

- Провести анализ предметной области и сформулировать функциональные требования.
- Спроектировать реляционную базу данных с учётом сложных связей между сущностями.
- Разработать серверную часть на базе Next.js App Router с REST API.
- Реализовать клиентский интерфейс с разграничением доступа по ролям (User, Developer, Admin).
- Обеспечить механизмы аутентификации, авторизации и защиты данных.
- Провести функциональное тестирование разработанной системы.

Объектом исследования являются процессы поддержки и продвижения программных продуктов (видеоигр) в цифровой среде.

Предмет исследования является архитектура, алгоритмы и программные средства информационной системы для поддержки indie-разработчиков.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

1. Анализ предметной области и обзор существующих решений для продвижения и поддержки игр

1.1. Анализ предметной области

Рынок цифровой дистрибуции видеоигр характеризуется высокой конкуренцией. По данным Steam, ежегодно публикуется более 10 000 новых игр, что создаёт проблему «видимости» для независимых разработчиков. Ключевые проблемы indie-разработчиков:

- Отсутствие прямой коммуникации с аудиторией — крупные платформы ограничивают возможности обратной связи.
- Сложность организации технической поддержки — отсутствие интегрированных инструментов трекинга багов.
- Монетизация раннего доступа — необходимость инструментов подписок и краудфандинга.
- Необходимость публикации контента — новости, гайды, обновления требуют отдельных каналов.

Анализ существующих решений показал отсутствие комплексной платформы, объединяющей каталог, поддержку, подписки и социальные функции в едином интерфейсе.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

1.2. Формулирование требований к проектируемой информационной системе

В таблице 1 представлены функциональные требования

Таблица 1 – Функциональные требования

ID	Требование	Роль
FR-01	Регистрация и аутентификация пользователей	Все
FR-02	Публикация игр с медиа-контентом (обложки, скриншоты, файлы)	Developer
FR-03	Просмотр каталога игр с фильтрацией по жанрам и платформам	Все
FR-04	Система подписок на игры (tier-модель: Bronze/Silver/Gold)	User
FR-05	Создание и обработка тикетов (bug/feature/question)	User / Developer
FR-06	Публикация новостей и гайдов	Developer / Admin
FR-07	Комментирование статей с поддержкой вложенных ответов	Все
FR-08	Личные сообщения между пользователями	Все
FR-09	Подписка на разработчиков (Follow)	User
FR-10	Управление пользователями и ролями	Admin
FR-11	Система уведомлений о событиях	Все
FR-12	Оценка и отзывы игр (1-5 звёзд)	User

1.3. Обоснование выбора технологического стека

Выбор фреймворка: Next.js 15 (App Router)

Next.js выбран по следующим причинам: - App Router — современная модель роутинга с серверными компонентами, позволяющая выполнять запросы к БД на стороне сервера без клиентского JavaScript. - Fullstack в одном

проекте — единая кодовая база для frontend и backend (API Routes). - SSR/SSG/ISR — гибкие стратегии рендеринга для оптимизации производительности. - Экосистема Vercel — оптимизированный деплой и CI/CD.

Выбор ORM: Prisma 5.22

Prisma обеспечивает: - Type-safe запросы — автогенерация типов на основе схемы. - Миграции — версионирование структуры БД через prisma migrate. - Prisma Studio — визуальный инструмент для работы с данными (порт 5555). - Поддержка SQLite для dev-режима с возможностью миграции на PostgreSQL.

Выбор аутентификации: iron-session

В отличие от JWT, iron-session: - Шифрует данные сессии в cookie (Seal), не требуя хранения состояния на сервере. - Не подвержен проблеме «утечки секретного ключа» JWT (токены нельзя отозвать). - Поддерживает secure и httpOnly флаги cookie.

Выбор стилизации: TailwindCSS

Утилитарный CSS-фреймворк обеспечивает: - Консистентный дизайн через конфигурацию темы. - Минимальный размер CSS-бандла (purge unused styles). - Быструю прототипирование интерфейсов.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

2. Проектирование архитектуры системы и разработка ER-модели базы данных

2.1. Разработка архитектуры базы данных

База данных спроектирована в виде реляционной модели с использованием Prisma Schema. Включает 20+ сущностей с нормализацией до 3НФ.

Ключевые сущности

User — центральная сущность системы.

Поля: id, email (unique), name, passwordHash, role (enum: user/developer/admin), isActive, avatarUrl, lastSeenAt, age, showFollowers. Имеет 15+ отношений: games, articles, newsPosts, tickets (созданные), assignedTickets (назначенные), comments, reviews, subscriptions, notifications, followers, following, conversations1/2, sentMessages.

Game — описание игрового продукта.

Поля: id, title, description, mediaUrl, gameFileUrl, requiredTier (null/bronze/silver/gold — уровень доступа), createdAt. Связи: author (User), tickets, articles, newsPosts, subscriptionTiers, reviews, media, genres (через GameGenre), platforms (через GamePlatform), files.

Ticket — система технической поддержки.

Поля: id, title, description, type (bug/feature/question), priority (low/medium/high), status (open/in_progress/resolved/closed), resolution, closeReason, createdAt. Связи: game, author, assignee (User, nullable).

SubscriptionTier / Subscription — модель монетизации. SubscriptionTier содержит name, price (Float), benefits, maxSlots, привязан к game. Subscription — конкретная активная подписка пользователя с startDate, endDate, status (active/cancelled/expired).

Comment — рекурсивная модель комментариев.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Поля: id, content, createdAt. Ключевая особенность — самореферентная связь: parentId → parent (Comment?), replies (Comment[]). Это позволяет строить дерево вложенных ответов неограниченной глубины.

Conversation / Message — система личных сообщений. Conversation содержит user1Id, user2Id с @unique([user1Id, user2Id]) — гарантия единого диалога между двумя пользователями.

Поля clearedByUser1/2 и deletedByUser1/2 реализуют мягкое удаление и очистку истории.

Таблица 2 - Many-to-many связи

Промежуточная таблица	Связь	Первичный ключ
GameGenre	Game ↔ Genre	@id([gameId, genreId])
GamePlatform	Game ↔ Platform	@id([gameId, platformId])
Follow	User ↔ User (follower/following)	@unique([followerId, followingId])

Уникальные ограничения

- GameReview: @unique([gameId, authorId]) — один отзыв на игру от пользователя.
- Conversation: @unique([user1Id, user2Id]) — единый чат между двумя пользователями.

Стратегии onDelete

- Cascade — для зависимого контента (медиа, отзывы, тикеты при удалении игры; сообщения при удалении диалога).
- SetNull — для необязательных связей (статья может потерять привязку к игре).

2.2 Разработка ER-модели базы данных

3. Программная реализация клиентской и серверной частей платформы.

3.1. Проектирование серверной части (Backend)

Архитектура API

Серверная часть реализована через Next.js App Router API Routes (src/app/api/.../route.ts). Иерархия включает 30+ эндпоинтов:

Аутентификация:

- POST /api/auth/register — регистрация с хешированием bcrypt.
- POST /api/auth/login — создание сессии iron-session.
- POST /api/auth/logout — удаление сессии. —
- GET /api/auth/me — получение текущего пользователя.
- POST /api/auth/change-password — смена пароля.

Контент:

- GET/POST /api/games — CRUD игр.
- GET/POST /api/articles — статьи и гайды.
- GET/POST /api/news — новостные посты.
- GET/POST /api/comments — комментарии (с поддержкой parentId).

Бизнес-логика:

- POST /api/subscribe — оформление подписки (с проверкой дубликатов).
- GET/POST /api/subscription-tiers — управление tier-ами (Developer).
- GET/POST /api/tickets — создание тикетов.
- PATCH /api/tickets/[id] — обработка тикетов (смена статуса, назначение).

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

- GET /api/developer/tickets — тикеты по играм разработчика.
- GET/POST /api/messages/conversation/[id] — чат.
- POST /api/upload — загрузка медиа-файлов.

Социальное:

- POST /api/user/follow — подписка на пользователя.
- GET /api/users/search — поиск пользователей.
- GET /api/notifications — уведомления.

Middleware и безопасность

src/middleware.ts — единая точка входа для всех HTTP-запросов(Рисунок 1):

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const response = NextResponse.next();
  response.headers.set('x-pathname', request.nextUrl.pathname);
  return response;
}
```

Рисунок 1 – единая точка входа для всех HTTP-запросов

Инъекция заголовка x-pathname позволяет клиентским компонентам определять текущий путь без useRouter() (недоступного в серверных компонентах App Router).

Ролевая защита реализована на уровне каждого API-роута:

```
if (!session.userId || (session.role !== 'developer' && session.role !== 'admin')) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

3.3. Проектирование клиентской части (Frontend)

App Router структура

src/app/

- |— page.tsx # Главная / каталог игр
- |— layout.tsx # Корневой layout с навигацией
- |— globals.css # Глобальные стили
- |— admin/ # Админ-панель
- |— api/ # API Routes
- |— developer/
 - | |— dashboard/ # Панель разработчика
 - | |— tickets/ # Тикеты по играм
 - | |— subscriptions/ # Управление tier-ами
- |— games/[id]/ # Карточка игры
 - | |— edit/ # Редактирование
 - | |— news/ # Новости игры
 - | |— subscribe/ # Подписка
- |— guides/ # База знаний
- |— knowledge/ # Статьи
- |— login/ # Авторизация
- |— messages/ # Личные сообщения
- |— news/ # Новостная лента
- |— profile/ # Личный кабинет
- |— register/ # Регистрация
- |— user/[id]/ # Публичный профиль

Компонентная архитектура

CRUD-компоненты:

- CreateGameForm
- CreateArticleForm

- CreateNewsForm
- CreateTicketForm
- EditGameButton
- DeleteGameButton

Медиа:

- CoverUploader
- MediaUploader
- GameGallery — загрузка и отображение медиа-контента.

Подписки:

- SubscribeButton
- TierSelect
- DeveloperSubscriptions — UI оформления и управления подписками.

Социальное:

- FollowButton
- CommentSection
- GameCommentSection
- GameReviewsSection
- SendMessageButton
- ExistingChatButton.

Тикеты:

- TicketSection
- DeveloperTickets
- QuickStats

Навигация и фильтры:

- Navigation
- GenreSelect
- GenreMultiSelect
- PlatformSelect.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

4. Функциональное тестирование, отладка и оптимизация работы системы.

Проверка корректности работы ключевых модулей: аутентификации, подписок, тикетов, комментариев, чата, загрузки файлов, ролевой защиты.

4.1. Тестовое окружение

Таблица 3 – Тестовое окружение

Параметр	Значение
ОС	Windows 11
Браузер	Google Chrome 124
Сервер	Next.js 15 dev (localhost:3000)
База данных	SQLite (dev.db)
ORM	Prisma Studio (localhost:5555)

4.2. Тест-кейсы и результаты

Таблица 4 - ТК Регистрации

Шаг	Действие	Результат
1	Открыть /register	Форма отображена
2	Заполнить поля, нажать "Зарегистрироваться"	Редирект на /login
3	Проверить БД	Запись в User с role="user"

Таблица 5 - ТК Аутентификации

Шаг	Действие	Результат
1	Войти с корректными данными	Сессия создана, редирект
2	Проверить /api/auth/me	Объект пользователя возвращён

Таблица 6 - ТК Создания игры (Developer)

Шаг	Действие	Результат
1	Войти как Developer	Успешно
2	Создать игру через CreateGameForm	Игра сохранена, обложка загружена

Таблица 7 - ТК Подписки на игру(User)

Шаг	Действие	Результат
1	Выбрать tier, нажать "Подписаться"	Подписка создана
2	Повторить попытку	Ошибка 400 "Already subscribed"

Таблица 8 - ТК Обработки тикетов

Шаг	Действие	Результат
1	User создаёт тикет	Статус "Открыт"
2	Developer меняет статус	"В процессе", уведомление отправлено
3	Закреть без причины	Ошибка 400

Таблица 9 - ТК Отправки комментария

Шаг	Действие	Результат
1	Написать комментарий	Отображается
2	Ответить на комментарий	Вложенность depth=1

Таблица 10 - ТК Чата

Шаг	Действие	Результат
1	Отправить сообщение	Диалог создан
2	Удалить диалог для себя	Скрыт для User1, видим для User2
3	User2 тоже удаляет	Физическое удаление

Таблица 11 - ТК Загрузки файлов

Шаг	Действие	Результат
1	Загрузить .jpg (5 МБ)	Успешно
2	Загрузить .exe	Ошибка 400 (тип)
3	Загрузить .png (15 МБ)	Ошибка 400 (размер)

Таблица 12 - ТК Ролевой защиты

Шаг	Действие	Результат
1	User делает POST /api/developer/games	403 Forbidden
2	Guest делает POST /api/subscribe	401 Unauthorized

4.3. Заключение по тестированию

Все 9 тест-кейсов пройдены успешно. Система корректно обрабатывает граничные случаи: повторные подписки, закрытие тикетов без обязательных полей, мягкое удаление диалогов, валидацию файлов. Ролевая модель надёжно защищает API-эндпоинты.

5. Расчет себестоимости разработки платформы.

Исполнитель: один разработчик (студент-программист).

Средняя ставка: 500 руб./час.

$З_{от} = 800 \text{ ч} \times 500 \text{ руб./ч} = 400\,000 \text{ руб.}$

5.1. Накладные расходы и итоговая себестоимость

Накладные расходы (амортизация оборудования, электроэнергия, лицензии ПО, доступ к сети) принимаются в размере 20 % от затрат на оплату труда:

$З_{нр} = 400\,000 \times 0,20 = 80\,000 \text{ руб.}$

Итоговая себестоимость:

$С = З_{от} + З_{нр} = 400\,000 + 80\,000 = 480\,000 \text{ руб.}$

5.2. Экономическая эффективность

Расчётная себестоимость разработки MVP платформы составляет 480 000 рублей. Данная оценка является базовой для студенческого проекта. Коммерческая реализация с учётом дизайна, DevOps, маркетинга и интеграции платёжных систем потребует дополнительных инвестиций в размере 200–300 % от базовой оценки.

Перспективы монетизации платформы:

Комиссия с подписок (10–15 %)

Платные продвижения игр в каталоге

Премиум-функции для разработчиков (расширенная аналитика)

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Заключение

В ходе выполнения проекта разработана полнофункциональная веб-платформа для продвижения и поддержки видеоигр. Реализованы все запланированные модули:

Аутентификация и авторизация на базе iron-session с тремя ролями пользователей.

- Каталог игр с фильтрацией, поиском, медиа-галереей и системой оценок.
- Монетизация через tier-based подписки с защитой от повторного оформления.
- Техподдержка с полным жизненным циклом тикетов, статусной машиной и уведомлениями.
- Контент — новости, гайды, статьи с рекурсивными комментариями.
- Коммуникация — личные сообщения с мягким удалением диалогов и подписки на разработчиков.
- Администрирование — управление пользователями, ролями и модерация.

Технологический стек — Next.js 15, Prisma 5.22, TypeScript, TailwindCSS, iron-session, SQLite — продемонстрировал свою эффективность для быстрого прототипирования и дальнейшего масштабирования.

Перспективы развития:

Интеграция платёжных систем (Stripe, ЮKassa) для реальных транзакций.

Миграция на PostgreSQL и Redis для production-нагрузок.

Real-time уведомления через WebSockets.

Мобильное приложение на React Native.

Система аналитики и дашбордов для разработчиков.

Система аналитики и дашбордов для разработчиков.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		